Auditable Residual Surrogates for Nonlinear DSGE Estimation

Anonymous

October 29, 2026

Abstract

Nonlinear solution can dominate DSGE estimation, but a surrogate is useful only if it preserves the target. We develop an auditable residual-surrogate workflow in which a neural network learns the gap between nonlinear stochastic extended-path trajectories and a parameter-specific first-order solution. In the complete 23-equation Galí model, Python and MacroModelling.jl agree on the steady state, first-order solution, five-period nonlinear scenario tree, and one-transition likelihood contribution. All eight reduced-horizon paths solve, but a difficult calibration fails at horizon six. On a paper-matched Smets–Wouters benchmark, CPU and GPU log likelihoods differ by 3.87×10^{-12} and all 36 gradient components by at most 5.43×10^{-10}. In the recorded software environments, the RTX 5090 deployment nevertheless takes 1.47 seconds per steady gradient versus 0.080 seconds on the CPU. The evidence establishes reduced-horizon algorithmic parity and an executable validation architecture, not a converged nonlinear posterior or GPU speed advantage.

1 Introduction

Likelihood-based estimation of a nonlinear DSGE model repeats a costly numerical experiment at every parameter proposal. A single evaluation may require a steady state, a local solution, nonlinear conditional trajectories, latent-shock inversion, and measurement-error evaluation. Posterior simulation then repeats this stack thousands of times. Approximation is attractive, but an unmeasured approximation can change the objective, the posterior geometry, and even the numerical support of the model.

This paper develops a residual-surrogate workflow designed around that risk. Its central contribution is a validation architecture for nonlinear DSGE surrogates. The architecture separates trajectory labels, the differentiable objective used for proposals, and the endpoint density that defines the posterior claim. This separation matters because a numerically correct HMC kernel can still target the wrong economic object.

The method solves the complete nonlinear equations at designed parameter, state, and shock inputs, then learns the difference between those solutions and a parameter-specific first-order reduced-order model (ROM). The ROM supplies a disciplined local baseline; the network is responsible only for the remaining nonlinearity. During sampling, the cheap differentiable objective supplies the Hamiltonian force. A separately declared density is evaluated at each proposal endpoint. Standard Metropolis correction then preserves that declared density when the numerical proposal map is reversible and volume preserving.

The preserved Galí pilots establish that this software path composes on the complete 23-equation nonlinear model while only the estimated parameter vector is reduced. Across 80 deterministic extended-path targets, the largest recorded equation residual is 9.92×10^{-9}. Python and JAX evaluations of the same surrogate profile objective differ by at most 1.71×10^{-13} at the audited

calibration. A calibration block shows a 53.9 percent mean reduction in state error relative to the ROM after per-output shrinkage is fitted on that same block. The last number is not an untouched test result, the HMC budgets are not convergence evidence, and no preserved run yet establishes a nonlinear posterior speed advantage.

The first stochastic positive-order pilot is deliberately retained despite failing its hard gate. Only three of eight parameter paths completed a ten-period, order-one, three-node calculation, while five paths failed at the initial simulated period. A later clean preflight isolates the solver from training and inference. All eight prespecified path evaluations, spanning seven unique policy-rule points because the reference is evaluated twice, pass the 10^{-8} residual gate at horizons three through five, but the hard design fails under both sequential and batched solvers at horizon six. Moreover, the horizon-four- to-five change reaches 3.96×10^{-4} for log output, 1.22×10^{-3} for annualized inflation, and 1.39×10^{-3} for the annualized policy rate. Residual convergence at horizon five therefore does not establish terminal-horizon convergence.

Independent cross-language comparisons close three scoped implementation risks. The Python and MacroModelling.jl calculations use byte-identical copies of the complete Galí source, inject the policy coefficients by name despite their different internal parameter orders, and retain the duplicated reference in the eight-case denominator. Every non-stochastic steady state and first-order Schur solution passes its frozen tolerance. A second comparison activates all three shock channels and matches the complete five-period, order-one sparse nonlinear scenario tree, rather than only its central path. A reference adapter then reconstructs one nonlinear inversion-likelihood contribution from the same model. Recovered shocks agree within 8.30×10^{-10}, the finite-difference observation Jacobians within 9.49×10^{-7}, and log likelihoods within 5.30×10^{-5}. This reduced-horizon, one-transition result does not establish multi-period filtering or production-horizon trajectory and likelihood parity.

An external empirical check reaches a different boundary. On the declared 1960Q1–2004Q4 Smets–Wouters sample, the exact CPU and GPU likelihoods agree within 3.87×10^{-12} and the largest 36-parameter gradient discrepancy is 5.43×10^{-10}. The GPU also matches an independent Dynare likelihood within 1.80×10^{-11}. Single-draw timing in the recorded environments does not favor the GPU: the steady exact gradient takes 1.47 seconds on an RTX 5090 D and 0.080 seconds on the Apple CPU. The empirical check validates the differentiable kernel but does not yet establish posterior convergence or effective-sample throughput.

The validation architecture has three components. It first distinguishes the nonlinear trajectories used as labels from the surrogate profile objective and the nonlinear likelihood required for a structural-posterior claim. It then integrates parameter-dependent solution, residual learning, bounded-parameter HMC, and explicit failure handling in a content-addressed workflow. Finally, it turns larger accuracy and efficiency claims into executable gates covering stochastic expectations, cross-language parity, untouched test accuracy, posterior fidelity, sampler diagnostics, and full computational accounting.

The paper does not claim that endpoint correction or neural solution methods are new by themselves. Its narrow claim is that nonlinear DSGE surrogates can be audited well enough to distinguish established algorithmic parity from unproved posterior and speed claims. Section 2 defines the economic and statistical objects. Sections 3 and 4 describe the surrogate and the target-corrected sampler. Sections 5 and 6 give the validation design and evidence, and Sections 7 and 8 state the decision rules and limitations.

1.1 Relation to the literature

The estimation problem sits between nonlinear DSGE solution, neural approximation, and accelerated MCMC. Extended path provides the nonlinear trajectory construction [Fair and Taylor, 1983]; stochastic extended path replaces future-shock means with numerical integration over conditional expectations [Adjemian and Juillard, 2026]. Likelihood construction for nonlinear DSGE models remains model- and solution-dependent [Kollmann, 2017], which is why this paper treats the trajectory solver, latent-shock treatment, and statistical objective as separate validation targets.

Neural networks have been used to solve high-dimensional dynamic economic models [Maliar et al., 2021]. Here the network does not replace the economic model wholesale: it learns the full-minus-first-order discrepancy and is always benchmarked against the ROM. Surrogate-gradient HMC and neural-gradient HMC already establish the proposal-side idea [Zhang et al., 2017, Li et al., 2019]. Our focus is the less forgiving DSGE case in which the endpoint density may contain parameter-dependent solution failures and may differ from the density used to generate the force. The independent implementation benchmark uses MacroModelling.jl [Kockerols, 2023], while the JAX path builds on the differentiable state-space perspective of Childers et al. [2022].

2 Economic model and statistical objects

Let $\theta \in \Theta$ denote structural parameters, s_t predetermined states, x_t endogenous variables, and ε_t structural shocks. Write the nonlinear equilibrium system as

$$F(x_{t-1}, x_t, \mathbb{E}_t x_{t+1}, \varepsilon_t; \theta) = 0. \tag{1}$$

For a given parameter vector and shock design, a nonlinear extended-path solver produces a trajectory $x^F_{1:T}(\theta)$ from the full-order model (FOM). A first-order solution produces the cheaper trajectory $x^R_{1:T}(\theta)$. We fit a network r_ψ to the residual

$$\Delta x_t(\theta) = x^F_t(\theta) - x^R_t(\theta), \qquad \widehat{x}^F_t(\theta) = x^R_t(\theta) + r_\psi(z_t, \theta), \tag{2}$$

where z_t contains the states and shocks available at the prediction point. Parameter vectors, rather than individual rows, are assigned to data partitions. This prevents periods from the same parameter-specific trajectory from appearing on both sides of a split.

The target nonlinear posterior for data $y_{1:T}$ is

$$\pi_F(\theta \mid y) \propto p(\theta) L_F(y_{1:T} \mid \theta), \tag{3}$$

where L_F denotes an independently validated nonlinear likelihood with a declared treatment of latent shocks and every required change-of-variables term. The present implementation instead defines a surrogate inversion objective $\widetilde{L}_\psi$ by replacing nonlinear trajectories with (2), optimizing latent shocks, and evaluating the shock and measurement penalties at that optimizer. Unless an integration, Laplace adjustment, or required change-of-variables term is supplied, this is a profile or pseudo-likelihood objective rather than a marginal likelihood. The two objects are therefore scientifically distinct. Agreement between Python and JAX evaluations of $\widetilde{L}_\psi$ is implementation parity; it is not evidence that $\widetilde{L}_\psi = L_F$.

More explicitly, let $h_\psi(s_t, \varepsilon_t, \theta)$ map a state and structural shock into the observables implied by the ROM plus residual surrogate. At each date, the implementation computes

$$\widehat{\varepsilon}_t(\theta) = \arg\min_{\varepsilon} \left\{ \|R^{-1/2}[y_t - h_\psi(s_t, \varepsilon, \theta)]\|_2^2 + \|Q^{-1/2}\varepsilon\|_2^2 \right\}, \tag{4}$$

Table 1: Numerical objects and the claims they can support

Object	Construction	Valid interpretation
Nonlinear label x^F	Extended-path solution of the complete equilibrium system	Supervised target for trajectory approximation; not a probability density
Surrogate objective $\widetilde{L}_\psi$	ROM plus learned residual, with shocks profiled by (4)	A declared pseudo-posterior target when combined with a prior and transform Jacobian
Nonlinear likelihood L_F	Independently validated statistical likelihood with latent variables and Jacobians treated explicitly	Structural likelihood and posterior in (3)

using a damped Gauss–Newton iteration, and then sums the Gaussian penalties evaluated at $\widehat{\varepsilon}_t(\theta)$. Here R and Q are diagonal in the pilot. The state passed to date $t+1$ follows the configured ROM or surrogate replay. Equation (4) makes the missing operation clear: the latent shocks are optimized, not integrated out. It also clarifies why agreement between two implementations of this calculation is a software check rather than likelihood validation.

Table 1 records the three numerical objects used below. A trajectory label is not a density, and a profile objective is not a marginal likelihood. Keeping their names distinct prevents a correct MCMC kernel from being used to support the wrong inferential claim.

The benchmark is the complete nonlinear New Keynesian system distributed with the MacroModelling.jl examples [Kockerols, 2023] and associated with Galí [2015]. It contains 23 equations, 16 declared parameters, and three structural shocks. The reduced experiment estimates the Taylor-rule responses to inflation and output, ϕ_π and ϕ_y, while retaining every model equation. Reducing the parameter dimension makes a numerical posterior reference feasible; it does not replace the model with the familiar three-equation linearization.

2.1 Pilot data and parameter design

The preserved software fixture contains ten synthetic observations of log output, annualized inflation, and the annualized nominal interest rate. The measurement standard deviation is 0.01 for each observable. The three declared structural shocks are technology, monetary-policy, and preference shocks, with model innovation scales 0.01, 0.0025, and 0.05 multiplying unit-variance innovations inside the equilibrium equations. These observations are useful for testing the inversion and sampling stack; they are not an empirical data application. The confirmatory experiment freezes a documented data-generating parameter, simulation seed, burn-in, initial condition, and shock path before training or posterior evaluation.

The deterministic default pilot contains eight parameter vectors and ten periods per vector. The reference calibration is included, and the two estimated policy coefficients are perturbed by the fixed design while the remaining 14 parameters stay at their declared calibration. Two complete parameter groups form the calibration block. The positive-order pilot uses the same reduced parameter design so that its purpose remains solver feasibility rather than a claim about broad parameter-space coverage.

3 Residual-surrogate construction

For training case i, let $u_i = (z_i, \theta_i)$ and $d_i = x_i^F - x_i^R$. Input and output location and scale are estimated only from the training partition. In standardized coordinates, the implemented data-fit loss is

$$\widehat{\psi} = \arg\min_{\psi} \frac{1}{\sum_{i \in \mathcal{I}_{\mathrm{tr}}} w_i} \sum_{i \in \mathcal{I}_{\mathrm{tr}}} w_i \left\| \widetilde{d}_i - r_\psi(\widetilde{u}_i) \right\|_2^2, \tag{5}$$

with zero weight for masked failures. The optimizer is AdamW with gradient clipping. The network embeds states and shocks, conditions hidden activations on the full estimated parameter vector through feature-wise affine modulation, and uses residual blocks with SiLU activations. Deployment restores the original units and adds the predicted discrepancy to the ROM.

The preserved default configuration uses one residual block, 64 hidden units, 400 epochs, batches of 32, and 64-bit training arithmetic. These choices are engineering settings, not selected hyperparameters. Confirmatory learning curves vary target count and network capacity using only training and calibration data; the untouched test partition is evaluated once for the frozen specification.

The implementation can fit an additional per-output shrinkage coefficient in $[0, 1]$ after network training. In the preserved pilots, those coefficients were fitted on the same parameter block used to report the resulting RMSE. That block is therefore a calibration set, not an untouched test set. We retain the values as engineering diagnostics but do not treat them as out-of-sample evidence. Publication launchers disable this calibration; a future use of shrinkage requires a distinct calibration partition and an untouched test partition.

3.1 Offline and online computations

The offline stage draws complete parameter vectors, solves a parameter-specific steady state and first-order ROM, and generates nonlinear extended-path targets conditional on designed state and shock paths. Each attempted target retains a case identifier, solver status, residual norm, and parameter-group identifier. Only the training partition determines feature scaling and network weights. Validation partitions are disjoint in complete parameter vectors, not merely in individual rows. The residual network and every preprocessing constant are then frozen and serialized with the model identity and source hashes.

The online stage transforms an unconstrained vector q to bounded structural parameters, adds the transform Jacobian to the declared log target, rebuilds the parameter-dependent objects required by the selected runtime, and evaluates either the profile objective or a nonlinear reference. A surrogate-gradient trajectory costs many cheap force evaluations but only one new target evaluation per transition. This accounting—rather than draws per second alone—is central when the endpoint target contains a nonlinear solution.

Extended path traces back to Fair and Taylor [1983]. Deterministic extended path sets future innovations to their conditional means and therefore does not capture Jensen effects in nonlinear expectations. Stochastic extended path approximates those conditional expectations over future shocks [Adjemian and Juillard, 2026]. This is why positive branching order, multiple nodes, and active variation in every shock are hard requirements for confirmatory evidence, even though order zero is useful for debugging deterministic nonlinearity.

4 Target-corrected surrogate trajectories

Let q be an unconstrained parameterization of θ, let p be Gaussian momentum with kinetic energy $K(p)$, and let $\pi_T(q)$ be the density the sampler is intended to preserve. A cheaper differentiable density $\pi_P(q)$ defines the force used in L leapfrog steps. Denote the resulting deterministic map by $(q', p') = \Phi_{\pi_P}(q, p)$. The acceptance probability is

$$\alpha\{(q,p),(q',p')\} = \min\left\{1, \exp\left[-H_T(q',p') + H_T(q,p)\right]\right\}, \quad H_T(q,p) = -\log \pi_T(q) + K(p). \quad (6)$$

Proposition 1 (Target invariance)**.** *Suppose Φ_{π_P} is reversible after momentum negation and volume preserving, and π_T is evaluated at both endpoints used in* (6)*. Then the Metropolis transition leaves π_T invariant, even when $\pi_P \neq \pi_T$.*

Proof. On the extended state $z = (q, p)$, the target density is proportional to $\exp\{-H_T(z)\}$. Compose the leapfrog map with momentum negation. By the assumptions, the resulting map is an involution with unit Jacobian determinant. The deterministic Metropolis ratio is therefore $\exp\{-H_T(z') + H_T(z)\}$, and (6) gives detailed balance on the extended state. Marginalizing the refreshed momentum leaves $\pi_T(q)$ invariant. The construction of Φ_{π_P} affects acceptance but not this invariance argument. □

One transition has four auditable steps. Draw fresh Gaussian momentum; apply L leapfrog steps using only $\nabla \log \pi_P$; reject immediately if the proposal or target is non-finite; otherwise apply (6) using π_T at the two endpoints. Step size and mass-matrix adaptation end with warmup. The frozen network, normalization constants, parameter transform, and target implementation are part of the recorded sampling state.

The proposition is the standard deterministic-proposal Metropolis argument for leapfrog HMC [Neal, 2011]. It places two practical restrictions on the implementation. First, the surrogate and every preprocessing transformation used in the force must be frozen during production sampling. Second, "exact" always modifies the declared target. If $\pi_T \propto p(\theta)\widetilde{L}_\psi$, the chain is exact for the surrogate-defined posterior, not for (3). Exact nonlinear posterior claims require endpoint evaluation of L_F itself.

For bounded parameters $\theta_j \in (a_j, b_j)$, the implementation samples $q_j \in \mathbb{R}$ and uses

$$\theta_j = a_j + (b_j - a_j)\,\mathrm{logit}^{-1}(q_j), \qquad \log J(q) = \sum_j \left[\log(b_j - a_j) + \log \sigma(q_j) + \log\{1 - \sigma(q_j)\}\right]. \quad (7)$$

Thus the endpoint target is defined in q coordinates and must include $\log J(q)$. Invalid steady states, indeterminate local solutions, failed nonlinear paths, or non-finite objective values must be rejected as points outside support. A large finite penalty is not equivalent: it creates an artificial plateau with positive density. The preserved pilot still uses such a sentinel and therefore cannot support an exact structural-posterior claim.

The sampler has three implementation-level regression checks. When proposal and target densities coincide, its draws and accept/reject decisions match the ordinary static-HMC implementation under the same key. Under a deliberately shifted and rescaled proposal force, a fixed-seed multi-chain test recovers the known moments of a standard-normal target after endpoint correction. Finally, non-finite candidate targets receive zero acceptance probability. These tests check kernel plumbing; they do not replace convergence and posterior-fidelity evidence for the DSGE application.

This distinction is especially relevant for differentiable state-space and DSGE estimation, where automatic differentiation improves computation but does not resolve approximation error in the target [Childers et al., 2022]. Related work has made nonlinear DSGE likelihoods more tractable

[Kollmann, 2017]; our evaluation therefore treats accuracy against an independently computed nonlinear reference as a primary outcome.

5 Validation design

The confirmatory design has two tiers. Tier 1 estimates (ϕ_π, ϕ_y) and uses a dense posterior grid. Grid resolution is doubled until posterior moments and interval endpoints stabilize. This tier isolates numerical correctness from high-dimensional MCMC behavior. Tier 2 estimates a prespecified four-to-six parameter subset and measures scaling with four independently initialized chains. The paper will report Tier 2 only after Tier 1 passes.

Tier 1 validates the estimator as a matched pipeline, not as a collection of independent numerical checks. Synthetic data are generated by the declared nonlinear reference at a frozen truth. Direct-reference and surrogate arms then use identical observations, priors, parameter transforms, shock treatment, objective terms, initializations, and chain seeds; only the transition evaluator changes. The residual must first pass a parameter-held-out trajectory test, then a common-shock inversion grid, and finally a matched HMC comparison. The final comparison reports interval overlap, truth coverage, and posterior-mean differences in combined Monte Carlo standard-error units, with every numerical failure retained. Agreement between two targets that both miss a weakly identified truth is recorded as an identification failure rather than surrogate success.

Training, validation, and posterior-region designs are disjoint in parameter vectors, shock sequences, and trajectories. Validation includes regular prior-interior points, posterior-region points, and a boundary-stress stratum. The last stratum is reported separately rather than deleted through a stability- margin screen. For stochastic extended path, evidence runs use positive branching order, multiple quadrature nodes, and nonzero coverage of every declared shock. Common case identifiers are evaluated by both the Python and MacroModelling.jl implementations.

For trajectory case i with d_x reported outputs, the primary approximation metric is the scale-adjusted root mean squared error

$$e_i^x = \left\{ \frac{1}{T d_x} \sum_{t=1}^{T} \left\| D_x^{-1} (\widehat{x}_{it}^F - x_{it}^F) \right\|_2^2 \right\}^{1/2}, \tag{8}$$

where D_x is fixed from the training partition. Paired differences $e_{i,\mathrm{ROM}}^x - e_{i,\mathrm{sur}}^x$ are summarized with a 95 percent interval over prespecified test cases. Reporting includes medians, upper-tail quantiles, and every output separately; an average cannot hide failure in one observable.

Likelihood geometry is compared after removing an arbitrary additive constant. For reference point θ_0, define

$$e^\ell(\theta) = \left[\log \widetilde{L}_\psi(\theta) - \log \widetilde{L}_\psi(\theta_0) \right] - \left[\log L_F(\theta) - \log L_F(\theta_0) \right]. \tag{9}$$

The grid analysis reports the distribution of e^ℓ, rank reversals near the mode, and normalized gradient discrepancies only where the reference gradient exceeds a frozen floor. Posterior means and interval endpoints are compared to their combined grid-discretization or Monte Carlo uncertainty, not to an arbitrary number of printed decimal places.

The primary accuracy outcomes are scaled equilibrium residuals; held-out, per-output trajectory errors; paired improvement over the ROM with uncertainty; centered log-likelihood error; rank reversals near the mode; and normalized gradient error away from zero-gradient cases. Posterior comparison reports differences relative to pooled Monte Carlo error, interval endpoint differences,

and a distributional diagnostic. Sampling reports rank-normalized split $\widehat{R}$, bulk and tail effective sample sizes (ESS), ESS per gradient evaluation, and ESS per second.

Timing uses the same workload, precision, and hardware for both methods. Compilation, host callbacks, device transfers, and training are reported separately. Warm likelihood microbenchmarks use repeated synchronized calls; end-to-end comparisons use independent processes. Continuous device telemetry records utilization, allocated memory, power, temperature, and clock rate; the summary reports upper-tail utilization, peak memory, and integrated energy. Because these are device-wide counters, confirmatory runs require a dedicated GPU. Sampling invokes `nvidia-smi`; both comparison arms use the same interval, and unprofiled sensitivity timings bound monitoring overhead. The current Schur QME path invokes a sequential SciPy host callback, so a GPU label does not imply a GPU-native likelihood. The available GPU-native matrix-sign alternative must pass Galí-specific solution and likelihood parity before it enters a speed claim.

The primary decision is sequential. Experiment A first asks whether the two-parameter nonlinear reference and surrogate conclusions agree within the frozen numerical tolerances. A failure stops escalation to the higher-dimensional sampling experiment. Conditional on correctness, Experiment B asks whether the method improves effective samples per second after offline target generation, training, compilation, transfers, and failed solves are included. A method can therefore pass the correctness gate and fail the computational-value gate.

5.1 External empirical validation on Smets–Wouters

The synthetic known-truth experiments are complemented by a paper-matched empirical benchmark based on the Smets–Wouters model studied by Farkas and Tatar [2023]. This benchmark is external to surrogate training: it uses the seven published observables, the declared 1960Q1–2004Q4 sample, all 36 estimated parameters, the archived Dynare bounds and prior families, and six presample periods. Both implementations solve the steady state at each parameter draw. A fixed-reference steady state cannot support either the parity or posterior claim.

Validation is again layered. First, float64 CPU and GPU evaluations at common parameter vectors must agree in log likelihood and gradient. Second, four-chain GPU sampling must pass divergence, tree-depth, split-$\widehat{R}$, and ESS gates. Third, the original-mode posterior is compared with the published means and 5–95 percent intervals using prespecified normalized errors and interval membership. Finally, the calculation is repeated from the archived Dynare initialization and the prior means. This dispersed initialization check is necessary because Farkas and Tatar [2023] find a second mode and emphasize that HMC cannot traverse well-separated modes by dynamics alone.

The empirical benchmark validates the differentiable linear state-space and sampling stack; it does not by itself validate the nonlinear SEP/ResNN likelihood. Conversely, the nonlinear trajectory tests do not substitute for the SW07 posterior replication. The replication package therefore reports the two evidence layers separately and promotes a claim only when the relevant layer's machine-readable gates pass. One archival ambiguity remains explicit: the paper declares a 180-quarter sample, whereas the data file contains 230 quarters and some archived command files specify `first_obs=1`. The confirmatory implementation follows the declared sample and matches Dynare's fixed-point likelihood within 1.80×10^{-11}. This check resolves the kernel and sample mapping, but a new sample-matched Dynare posterior is still required before the archived posterior table is described as replicated.

The protocol is frozen before confirmatory execution. It records the repository revision, dirty-worktree state, environment, command, seeds, hardware, precision, and content hashes for every evidence artifact. Numerical failures remain in attempted-case denominators and are stratified by prior-interior, posterior-region, and boundary-stress designs. Machine-readable evidence is the

Table 2: Preserved full-model Galí engineering pilots

Run	Target runtime	$N_\theta \times T$	Max. residual	Calib. RMSE	Corrected	Accept.	Draws/s
Quick smoke	fixed-reference	4×4	7.21e-09	9.17e-05	No	1.000	0.378
Corrected resource	full-jax	4×4	7.21e-09	9.17e-05	Yes	1.000	0.0267
Default pilot	fixed-reference	8×10	9.92e-09	7.86e-06	No	1.000	8

Notes: All runs use the complete 23-equation nonlinear Galí source but estimate only two policy-rule parameters. N_θ is the number of parameter vectors and T is the observation horizon. Target construction used branching order zero and one node, so these are deterministic nonlinear extended-path pilots, not stochastic-expectations SEP validation. The reported calibration RMSE is computed on the parameter block also used to fit per-output residual-shrinkage factors; it is not an untouched test error. The archived likelihood payload also duplicated equation-level innovation scales in the shock prior. "Full-JAX" recomputes the parameter-dependent steady state and first-order ROM inside the surrogate profile inversion objective. It does not evaluate a full nonlinear SEP likelihood inside HMC. Correction therefore targets the declared surrogate objective, not an exact full-nonlinear posterior. The quick, corrected-resource, and default runs use respectively 1, 2, and 1 chains with 2, 2, and 50 post-warmup draws per chain. These runs establish execution and resource feasibility only.

Table 3: Archived Galí GPU execution profile and telemetry boundary

Run	Runtime	Endpoint evals	Pipeline min	Draws	HMC s	Draws/s	Start % / MiB
Quick smoke	fixed-reference	–	2.12	2	5.29	0.3783	0 / 260
Corrected resource	full-jax	8	2.13	4	149.66	0.02673	0 / 260
Default pilot	fixed-reference	–	22.62	50	6.25	8.001	0 / 260

Notes: All rows report JAX 0.11.1 on cuda:0 in 64-bit mode. The archived device reports 32,760 MiB, NVIDIA driver 595.71.05, and CUDA 13.2. "Pipeline min" is the recorded target-generation and training stage, not a common end-to-end workload. HMC draw counts, chains, likelihood runtimes, trajectory budgets, and endpoint evaluations differ across rows. The final column is a single `nvidia-smi` snapshot taken at run startup; it is not an average or peak. Consequently these data establish GPU execution and document a high corrected-run burden, but they support no utilization, memory-capacity, energy, or speedup claim. The replication code now provides continuous dedicated-device sampling for future same-work runs.

source of manuscript tables; hand-entered numerical cells are not permitted.

6 Computational evidence

The strongest current result is independent algorithmic parity on the complete 23-equation model. Python and MacroModelling.jl agree on the non-stochastic core, the complete five-period stochastic scenario tree, and one nonlinear inversion-likelihood contribution. The same evidence also identifies the boundary of that result: the hard path fails at horizon six, and neither a multi-period likelihood nor a posterior has yet passed an independent gate.

Table 2 establishes three narrow engineering facts. The parser, parameter-specific preflight, nonlinear equation solver, residual network, likelihood, and sampler compose on the full model. The order-zero extended-path targets satisfy a strict residual check in this small design. Finally, the Python and JAX implementations of the same surrogate profile inversion objective agree at the audited calibration. The recorded improvement over the ROM is a calibration-set result because per-output shrinkage was fitted on the same block; it is not counted as a fourth out-of-sample fact.

The corrected resource run establishes only that endpoint evaluation is possible under a tiny budget. It retained four draws in total, achieved 100 percent acceptance, and processed 0.0267 draws per second. Those numbers are not convergence evidence and are not a speed advantage. The acceptance rate instead suggests that the static-HMC tuning is far from an efficient production regime.

Table 3 separates GPU execution evidence from a GPU performance claim. All three archived

Table 4: Matched SW07 CPU–GPU likelihood and gradient profile

Metric (seconds)	CPU	GPU	CPU/GPU
Likelihood first call	7.433	44.907	0.17
Likelihood steady median	0.015	0.065	0.22
Gradient first call	330.261	290.327	1.14
Gradient steady median	0.080	1.472	0.05

Notes: Both backends evaluate the same float64, 36-parameter likelihood on the 180-quarter sample, solve the steady state at every draw, and use theoretical Kalman initialization. First-call times include JAX/XLA compilation; steady medians exclude compilation. CPU/GPU is the ratio of elapsed times, so values above one favor the GPU. The likelihood discrepancy is 3.87×10^{-12}; the maximum gradient-component discrepancy is 5.43×10^{-10}. The CPU environment is macOS arm64 with JAX 0.4.30 and NumPyro 0.19.0; the GPU environment is Linux x86_64 on an RTX 5090 D with JAX 0.11.1 and NumPyro 0.21.0. The timing ratio is therefore an observational deployment comparison, not a controlled hardware-only ablation. These microprofiles do not measure effective samples per second.

Table 5: First provenance-captured stochastic extended-path pilot

Metric	Recorded value
SEP order; nodes; horizon	1; 3; 10
Complete parameter paths	3/8 (37.5%)
Accepted targets	30/80
Accepted-target maximum residual	9.41e-09
Pipeline time	119.6 minutes
Same-object Python/JAX difference	3.10e-12
HMC smoke: chains; warmup; retained	1; 25; 50
HMC smoke: acceptance; draws/s	100.0%; 7.24
Strict quality gate	failed

Notes: Five designs, including the reference path, failed at the first simulated period. The recorded RMSE used shrinkage calibrated on the reported block, and the archived likelihood payload applied shock scales twice. The run therefore diagnoses numerical feasibility only; it is not estimation evidence.

pilots used the CUDA backend in 64-bit mode on the same recorded device environment. The parameter-dependent corrected run required 149.7 seconds for four retained draws and eight endpoint target evaluations, whereas the fixed-reference runs used different objectives, budgets, and chain configurations. The contrast motivates treating endpoint correction as a bottleneck candidate but is not a same-work comparison. Moreover, the archived `nvidia-smi` record is a single run-start snapshot. Its zero utilization and 260 MiB allocation describe initialization, not the workload. They cannot identify peak memory, average utilization, or energy. The supplied continuous profiler records those quantities for the frozen confirmatory runs; no acceleration claim is made until repeated same-work baseline and treatment runs are available.

Table 4 reports the first matched cross-device profile on the empirical SW07 likelihood. Numerical parity is tight: CPU and GPU log likelihoods differ by 3.87×10^{-12}, and the largest discrepancy across 36 gradient components is 5.43×10^{-10}. The timing result is less favorable to the GPU. It shortens gradient compilation by about 12 percent, but steady single-draw execution is 4.47 times slower for the likelihood and 18.5 times slower for its gradient. The CPU and GPU runs use different recorded JAX/NumPyro versions, so this is an observational deployment comparison rather than a controlled hardware-only ablation. It rules out an acceleration claim for the tested deployment, not for GPUs generically. Vectorized-chain ESS per second is a separate gate and remains unreported

Table 6: Reduced-horizon stochastic SEP evidence

SEP horizon	Accepted paths	Maximum residual	Newton iterations
3	8/8	1.73×10^{-12}	5–5
4	8/8	7.44×10^{-9}	4–5
5	8/8	1.93×10^{-10}	5–7
Horizon comparison	**All-state max**	**$\log Y$ max**	**π^{ann} / i^{ann} max**
3→4	0.0137	0.00029	0.000968 / 0.00112
4→5	0.0212	0.000396	0.00122 / 0.00139
3→5	0.0349	0.000686	0.00219 / 0.00251

Notes: Horizons three through five use eight prespecified path evaluations spanning seven unique points in a 12 percent perturbation design for the two policy-rule parameters; the reference calibration is evaluated twice. The calculation uses order-one stochastic extended path, three Gauss–Hermite nodes, and an L_∞ residual gate of 10^{-8}. The horizon-six hard design fails under both the sequential (1.02×10^{-4}) and batched (1.02×10^{-4}) solvers. First-step changes show that residual convergence is not terminal-horizon convergence. This table supports a reduced numerical pilot only; it is not likelihood or posterior evidence.

Table 7: Independent Julia–Python core parity on the complete nonlinear Galí source

Gate	Observed	Threshold	Result
Joint design evaluations	8/8	8/8	Pass
Steady-state maximum absolute difference	9.06×10^{-14}	10^{-10}	Pass
First-order Schur maximum absolute difference	2.89×10^{-11}	10^{-8}	Pass
Python steady-state residual norm	1.11×10^{-16}	10^{-10}	Pass
Clean source revisions	2/2	2/2	Pass

Notes: Eight evaluations cover seven unique (ϕ_π, ϕ_y) points because the reference calibration is repeated at indices 0 and 4. The local and upstream model files share SHA-256 `53f909976abc9624...`. This gate covers the non-stochastic steady state and first-order Schur solution only; nonlinear trajectory, likelihood, posterior, and GPU claims remain open.

until the sampling run passes convergence diagnostics.

Table 5 records a useful negative result from the first provenance-captured positive-order run. With branching order one, three quadrature nodes, and a ten-period planning horizon, only three of eight parameter designs completed all ten periods. Five designs—including the reference calibration—failed the nonlinear solve at the first simulated period. The 30 accepted targets satisfied the residual tolerance, but the strict full-path gate correctly failed. The run took 119.6 minutes, roughly 12 minutes per simulated period, on an RTX 4080 SUPER. This combination of localized first-period failures and high dense-QR cost rejects a simple "more GPU memory is sufficient" interpretation. It instead motivates continuation methods, solver-level early stopping, and a frozen correctness study before any large posterior exercise.

The clean reduced-horizon preflight provides a narrower positive result. Horizons three, four, and five accept all eight path evaluations across seven unique parameter points; the maximum residual at horizon five is 1.93×10^{-10}, and the solver takes five to seven Newton iterations. Table 6 also shows why this is a pilot rather than a terminal-horizon validation. First-step outcomes continue to move as the horizon grows, and the difficult design fails at horizon six with residuals near 1.02×10^{-4} under both solver implementations. The evidence supports stochastic target construction at a reduced horizon. It does not support a nonlinear likelihood, posterior agreement, or a computational advantage.

Table 7 establishes the first independent subgate on the complete model. Both repositories

Table 8: Independent Julia–Python nonlinear SEP parity on the complete Galí source

Gate	Observed	Threshold	Result
Joint design evaluations	8/8	8/8	Pass
Complete scenario-tree maximum absolute difference	6.40×10^{-8}	10^{-6}	Pass
Expected-path maximum absolute difference	6.50×10^{-9}	10^{-8}	Pass
Python solver residual norm	5.63×10^{-11}	10^{-10}	Pass
Julia solver residual norm	8.50×10^{-11}	10^{-10}	Pass
Clean source revisions	2/2	2/2	Pass

Notes: The test uses a five-period, order-one sparse tree with three Gauss–Hermite nodes per shock, group counts $(1, 1, 9, 9, 9, 9)$, and first-period standardized innovations $(0.12, -0.08, 0.10)$ across all three shock channels. Eight evaluations cover seven unique (ϕ_π, ϕ_y) points; indices 0 and 4 repeat the reference calibration and reproduce bit-for-bit within each implementation. This closes only the reduced-horizon nonlinear-tree subgate; production-horizon likelihood, posterior, and GPU gates remain open.

Table 9: Independent one-transition Julia–Python nonlinear SEP likelihood parity

Gate	Observed	Threshold	Result
Recovered-shock maximum absolute difference	8.30×10^{-10}	10^{-7}	Pass
Jacobian maximum absolute difference	9.49×10^{-7}	10^{-5}	Pass
Log-likelihood absolute difference	5.30×10^{-5}	10^{-4}	Pass
Python inversion residual norm	1.22×10^{-10}	10^{-9}	Pass
Julia inversion residual norm	1.08×10^{-10}	10^{-9}	Pass
Clean source revisions	2/2	2/2	Pass

Notes: The observation is generated from standardized innovations $(0.02, -0.01, 0.015)$ in the complete 23-equation Galí model. Both adapters invert the same $(\log Y, 4 \log \Pi, 4 \log R)$ transition using a five-period, order-one sparse SEP solve, three Gauss–Hermite nodes per shock, a 10^{-12} nonlinear-solver tolerance, and the same forward-difference rule. The Python public likelihood agrees with its instrumented audit call within 10^{-8}. This closes only the one-transition likelihood subgate; multi-period recursion, production-horizon likelihood, posterior, and GPU gates remain open.

were clean and content addressed, the local and upstream Galí files had the same SHA-256 digest, and all eight evaluations passed. The worst steady-state discrepancy was 9.06×10^{-14}, while the worst first-order Schur-matrix discrepancy was 2.89×10^{-11}. These errors are well below the frozen tolerances, but the result concerns a non-stochastic steady state and local first-order solution.

Table 8 extends the independent check to nonlinear stochastic extended path at a frozen five-period horizon. Both implementations solve the complete sparse scenario tree with group counts $(1, 1, 9, 9, 9, 9)$ and all three first-period shock channels active. All eight evaluations pass. The worst complete-tree and expected-path discrepancies are 6.40×10^{-8} and 6.50×10^{-9}, respectively, while both implementations remain below the 10^{-10} residual gate. Repeating the reference calibration at indices 0 and 4 produces identical trees within each implementation. This result closes the reduced-horizon nonlinear-tree subgate only. It neither resolves the observed horizon-five-to-six instability nor, by itself, validates a nonlinear likelihood or posterior.

Table 9 adds the first direct nonlinear likelihood subgate. Because the pinned upstream public likelihood interface does not expose SEP controls or per-period nonlinear likelihood contributions, a separately specified Julia adapter reconstructs one inversion contribution from the upstream SEP solver. Both implementations invert the same synthetic transition in three observables and recover all three standardized innovations. The recovered-shock, finite-difference Jacobian, and log-likelihood discrepancies are 8.30×10^{-10}, 9.49×10^{-7}, and 5.30×10^{-5}, respectively. This closes a one-transition algorithmic parity check. It does not test state propagation across observations, a

Table 10: Hard gates for a production structural-posterior and speed claim

Evidence family	Required result
Scope and provenance	Full 23-equation source verified by content hash and upstream revision; frozen synthetic-data generator and estimand; clean repository revision; complete environment, data, command, seed, and artifact manifests.
Nonlinear solution	Positive-order stochastic SEP with multiple nodes and all shocks active; raw attempted and successful counts; scaled residuals; regular, posterior, and boundary-stress strata.
Independent parity	Common case identifiers evaluated in Python and MacroModelling.jl; both failures retained in the denominator; QZ, trajectory, and likelihood errors reported on common successful cases.
Surrogate accuracy	Disjoint parameter, shock, and trajectory partitions; shrinkage disabled or fitted on a separate calibration partition; untouched-test per-output and tail errors; paired improvement over ROM with a 95 percent interval; learning-curve and out-of-distribution results.
End-to-end pipeline	Known-truth nonlinear data; matched direct and surrogate data, priors, transforms, objective terms, and chain seeds; held-out residual, inversion-grid, and matched-HMC layers passed sequentially; failures retained.
Posterior fidelity	Two-parameter full nonlinear grid with resolution-doubling convergence; interval overlap and truth coverage for every identified parameter; posterior-mean differences no larger than 0.5 combined MCSE in the publication run.
Sampling	Four tuned chains for the scaling experiment; rank-normalized split $\widehat{R}$, bulk and tail ESS, failures, ESS per gradient evaluation, and ESS per second.
Timing	Same-work, same-precision comparison; synchronized warm calls and independent end-to-end processes; compilation, transfers, training, and host callbacks reported separately; continuous dedicated-device traces for utilization, peak memory, power, and energy.

Notes: These are claim-specific design requirements. Prespecified thresholds and case lists must be frozen before confirmatory execution. Passing them supports the reduced Galí experiment only; it does not establish universal accuracy across DSGE models.

production horizon, a likelihood surface over parameters, or a posterior.

7 Claim-specific decision rules

Table 10 defines the additional evidence needed to promote the present parity result into a production structural-posterior and speed claim. Counts are protocol defaults, not universal publication standards; learning curves, uncertainty intervals, and reference convergence determine whether they are sufficient.

All thresholds and case identifiers will be frozen before the final runs. The evidence builder reads raw result files, verifies the full-model identity, hashes its sources, and emits both machine-readable and manuscript inputs. A failed hard gate remains visible in the replication package; it is not converted into a missing observation or silently omitted path.

8 Limitations and interpretation

The preserved computational evidence remains claim-scoped. The earlier nonlinear targets use branching order zero and one node, so they do not validate stochastic expectations. The later

order-one, three-node evidence establishes target feasibility only through a five-period terminal horizon; first-step sensitivity and the horizon-six failure prevent a terminal-convergence claim. Most likelihood calls hold the reference steady state and ROM fixed. The reported surrogate RMSE values are calibration-adjusted rather than measured on an untouched test partition. The parameter-dependent full-JAX run still targets the surrogate profile inversion objective, assigns a finite -10^{12} sentinel to failed solves, and uses a host-callback Schur solve. Its archived payload also duplicated the equation-level innovation scales in the inversion prior; future evidence uses unit-variance standardized innovations. An exact structural posterior instead requires explicit support handling and an economically defined likelihood. The sampler budgets are too small for convergence diagnostics, and the data are synthetic and short. The independent nonlinear Julia comparison is restricted to a five-period sparse tree, a single frozen first-period shock vector, and seven unique policy-rule points. It does not establish production-horizon trajectory parity. At the pinned upstream revision, the public likelihood interface exposes neither SEP controls nor per-period nonlinear likelihood contributions. The reference adapter therefore validates only one inversion contribution; multi-period filtering, parameter-surface likelihood parity, and production-horizon likelihood parity remain open. The current evidence therefore supports neither exact nonlinear posterior sampling nor an end-to-end speedup claim. The archived GPU telemetry is likewise limited to run-start snapshots; continuous traces will enter the evidence record only from future dedicated-device runs.

Most importantly, the current evidence validates components rather than the complete estimator. The matched known-truth pipeline—held-out residual, common-shock inversion grid, direct-versus-surrogate HMC, interval overlap, truth coverage, and combined-MCSE comparison—has not yet been completed in this implementation. Until that gate passes, GPU execution establishes resource feasibility and profiling infrastructure, not a validated estimation result.

These limitations determine the next experiments rather than merely qualifying the conclusion. The present methods result is the validation architecture and the reduced-horizon parity evidence. A structural-posterior claim requires the two-parameter reference experiment to pass before the posterior dimension is increased. If the surrogate does not preserve the grid posterior, or if full accounting does not improve ESS per second, that negative result remains part of the evidence and the production claim fails.

9 Conclusion

Residual learning is a plausible way to accelerate nonlinear DSGE estimation: the first-order model supplies a disciplined baseline, and the network learns only the nonlinear discrepancy. The statistical payoff, however, depends on the endpoint density rather than on the quality of the proposal force alone. The complete Galí implementation passes independent steady-state, first-order, five-period nonlinear-tree, and one-transition likelihood gates. It also produces an informative negative result. Outcomes remain sensitive to the terminal horizon, and a difficult design fails at horizon six.

The immediate implication is methodological. Cross-language parity should be established in layers before an expensive posterior run: core solution, nonlinear tree, likelihood contribution, multi-period recursion, and only then posterior and same-work timing. The first three component layers pass here. Multi-period recursion, the matched known-truth estimator pipeline, and same-work timing remain open. This ordering makes the next claims falsifiable and keeps a failed accuracy or speed gate in the scientific record instead of redefining success after computation.

Data and code availability

The replication package accompanying this draft contains the model source, validation programs, tests, frozen tolerances, raw numerical artifacts, and machine-generated manuscript tables. Each evidence builder verifies source and artifact hashes before emitting a table. The package README maps the archived results to the commands that rebuild them.

Disclosure on computational assistance

Automated coding and language tools were used to inspect software, generate tests, and edit the working draft. Every numerical claim in the paper is linked to a machine-readable source file and must be verified by the authors. The final submission will follow the target journal's disclosure policy.

References

Stéphane Adjemian and Michel Juillard. Stochastic extended path. *Journal of Economic Dynamics and Control*, 182:105227, 2026. doi: 10.1016/j.jedc.2025.105227.

David Childers, Jesús Fernández-Villaverde, Jesse Perla, Christopher Rackauckas, and Peifan Wu. Differentiable state-space models and Hamiltonian Monte Carlo estimation. Working Paper 30573, National Bureau of Economic Research, 2022.

Ray C. Fair and John B. Taylor. Solution and maximum likelihood estimation of dynamic nonlinear rational expectations models. *Econometrica*, 51(4):1169–1185, 1983. doi: 10.2307/1912057.

Matyas Farkas and Balint Tatar. Bayesian estimation of DSGE models with Hamiltonian Monte Carlo. Technical report, University of Surrey, June 2023. URL `https://www.matyasfarkas.eu/publication/hmc/hmc.pdf`.

Jordi Galí. *Monetary Policy, Inflation, and the Business Cycle: An Introduction to the New Keynesian Framework and Its Applications*. Princeton University Press, second edition, 2015.

Thore Kockerols. MacroModelling.jl: A Julia package for developing and solving Dynamic Stochastic General Equilibrium models. *Journal of Open Source Software*, 8(89):5598, 2023. doi: 10.21105/joss.05598.

Robert Kollmann. Tractable likelihood-based estimation of non-linear DSGE models. *Economics Letters*, 161: 90–92, 2017. doi: 10.1016/j.econlet.2017.08.027.

Lingge Li, Andrew Holbrook, Babak Shahbaba, and Pierre Baldi. Neural network gradient Hamiltonian Monte Carlo. *Computational Statistics*, 34(1):281–299, 2019. doi: 10.1007/s00180-018-00861-z.

Lilia Maliar, Serguei Maliar, and Pablo Winant. Deep learning for solving dynamic economic models. *Journal of Monetary Economics*, 122:76–101, 2021. doi: 10.1016/j.jmoneco.2021.07.004.

Radford M. Neal. MCMC using Hamiltonian dynamics. In Steve Brooks, Andrew Gelman, Galin L. Jones, and Xiao-Li Meng, editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman and Hall/CRC, 2011.

Cheng Zhang, Babak Shahbaba, and Hongkai Zhao. Hamiltonian Monte Carlo acceleration using surrogate functions with random bases. *Statistics and Computing*, 27(6):1473–1490, 2017. doi: 10.1007/s11222-016-9699-1.